

MBA749 — Social Media Mining: Study Guide (Weeks 1-6)

Course: **Social Media Mining (SMM)**, IIT Kanpur MBA749. Textbook reference: socialmediamining.info.

This guide consolidates the slide decks and datasets uploaded for Weeks 1-6 into one set of study notes: graph fundamentals → spanning trees → centrality measures → similarity/ clustering → community detection → recommender systems.

Table of Contents

1. [Week 1 — Graph Essentials](#)
 2. [Week 2 — Components, Shortest Paths, Special Structures, MST](#)
 3. [Week 3 — Centrality Measures I \(Degree, Eigenvector, Katz, PageRank\)](#)
 4. [Week 4 — Centrality Measures II \(Betweenness, Closeness, Cliques\) & Similarity](#)
 5. [Week 5 — Community Detection & Louvain Modularity](#)
 6. [Week 6 — Recommendation in Social Media \(Content-Based Filtering\)](#)
 7. [Dataset Index](#)
-

Week 1 — Graph Essentials

Chapter 2 of the textbook: “Graph Essentials.”

Motivating example — Seven Bridges of Königsberg

Two islands + mainland connected by 7 bridges; the challenge is to find a walk crossing every bridge exactly once. Euler modeled the city as a graph and proved that, except for a walk’s start/end nodes, every node must be entered and left an equal number of times — so every other node needs **even degree**. Königsberg’s graph fails this test, so no such walk exists. This is the founding example of graph theory.

Networks = Graphs with meaning

Running example used throughout the course: a Twitter-style network of individuals with **propagation costs** on edges — “find the minimum cost to disseminate information to all individuals.” This is the same 9-node weighted graph shipped as `week1/nodes.csv` + `week1/edges.csv`, and it foreshadows Week 2’s Minimum Spanning Tree material.

Core definitions

- **Nodes / actors / vertices** (points) and **edges / ties** (connections). Node set $V = \{v_1, \dots, v_n\}$, size $|V| = n$; edge set $E = \{e_1, \dots, e_m\}$, size $|E| = m$.
- Naming is context-dependent: web graph nodes = “sites”; social graph nodes = “actors.”
- **Directed edges (arcs)** vs. **undirected edges**; an edge is written $e(v_i, v_j)$.
- **Neighborhood** $N(v)$: the set of nodes connected to v . In directed graphs this splits into $N_{in}(v)$ and $N_{out}(v)$.
- **Degree** d_i : number of edges at node i (= size of its neighborhood). Directed graphs: **in-degree** d_i^{in} , **out-degree** d_i^{out} .
- **Theorem 1**: $\sum_i d_i = 2|E|$ (undirected graph — every edge is counted once at each endpoint).
- **Lemma 1**: the number of odd-degree nodes in any graph is always even.
- **Lemma 2**: in any directed graph, $\sum_i d_i^{out} = \sum_j d_j^{in}$.

- **Degree distribution:** $\pi(d) = \{d_1, \dots, d_n\}$ is the degree sequence; $p_d = n_d/n$ where n_d is the number of nodes with degree d ; $\sum_{d=0}^{\infty} p_d = 1$.
- **Subgraph** $G'(V', E')$ of $G(V, E)$: $V' \subseteq V$, $E' \subseteq (V' \times V') \cap E$.

Graph representations

1. **Adjacency matrix** (sociomatrix): $A_{ij} = 1$ if there's an edge between v_i, v_j , else 0. Diagonal entries = self-loops. Social media graphs are typically **very sparse**. Directed graphs are generally asymmetric ($A \neq A^T$); undirected graphs are symmetric ($A = A^T$).
2. **Adjacency list:** per node, the list of nodes it connects to.
3. **Edge list:** a flat list of (u, v) pairs.

Types of graphs

- **Null graph** ($V = E = \emptyset$) vs. **empty/edge-less graph** ($E = \emptyset$, V non-empty); a null graph is a special case of an empty graph.
- **Simple graph** (at most one edge per node pair) vs. **multigraph** (multiple edges/loops allowed; adjacency matrix entries can exceed 1).
- **Weighted graph** $G(V, E, W)$: $A_{ij} = w_{ij} \in \mathbb{R}$, or 0 if no edge (e.g., airport distances).
- **Signed graph:** binary $\pm$ weights — represents friend/foe or social status.
- **Webgraph:** a directed multigraph; nodes = sites, edges = hyperlinks; parallel links and self-loops allowed.

Connectivity & paths

- **Adjacent nodes** (joined by an edge) vs. **incident edges** (share an endpoint; in directed graphs, directions must line up).
- **Walk:** a sequence of incident edges traversed in order. **Open** (doesn't return to start) vs. **closed** (does). Length = number of edges traversed.
- **Path:** a walk with all distinct nodes/edges. A **closed path = cycle**.
- **Random walk:** the next node is chosen randomly among neighbors; edge weights can define transition probabilities, with $\sum_x w_{i,x} = 1$ over edges leaving v_i .
- **Connectivity:** v_i is connected/reachable to v_j if adjacent or a path exists. A graph is **connected** if a path exists between every node pair. For directed graphs: **strongly connected** (a directed path exists both ways between every pair) vs. **weakly**

connected (a path exists once direction is ignored). Otherwise the graph is **disconnected**.

Week 2 — Components, Shortest Paths, Special Structures, MST

Continuation of Chapter 2.

Components

- A **component** (undirected graph) is a connected subgraph — a path exists between every pair of nodes inside it.
- Directed graphs: a **strongly connected component** requires a path $u \rightarrow v$ and $v \rightarrow u$ for every pair. A **weakly connected component** only requires connectivity once directions are dropped.

Shortest paths & diameter

- **Shortest path** length between v_i, v_j is denoted $l_{i,j}$.
- **n-hop neighborhood**: generalizes neighborhood using shortest-path distance $\leq n$.
- **Diameter**: the length of the longest shortest path between any node pair in the graph.

Adjacency matrix & path counting

- $A_{ij} = 1$ if there's an edge $i \rightarrow j$.
- Number of length-2 paths from i to j via any node k : $N_{ij}^2 = \sum_k A_{ik}A_{kj} = (A^2)_{ij}$.
- Number of length-3 paths: $N_{ij}^3 = \sum_{k,l} A_{ik}A_{kl}A_{lj} = (A^3)_{ij}$.
- Number of **common neighbors** of i, j = entry $[i, j]$ of $A \times A^T$ ($= A^2$ for the undirected/symmetric case).

Special graph structures

- **Tree**: undirected, acyclic, exactly one path between any node pair; $|V| = |E| + 1$. A set of disconnected trees is a **forest**.
- **Spanning tree**: a subgraph of a connected graph that is a tree touching every node. A graph can have multiple spanning trees. In a weighted graph, a spanning tree's weight is the sum of its edge weights; the lowest-weight one is the **Minimum Spanning Tree (MST)**.

- **Complete graph**: every possible node pair is connected.
- **Planar graph**: can be drawn with no edge crossings (except at shared endpoints).
- **Bipartite graph**: nodes split into two parts such that every edge has one endpoint in each part.
 - **Affiliation networks** are bipartite (people $\leftrightarrow$ organizations, users $\leftrightarrow$ items via like/share/purchase/rating).
 - Example: people $\leftrightarrow$ corporate board membership.
 - **One-mode projection** via membership matrix X : XX^T = similarity between people; X^TX = similarity between groups/items. Diagonal entries = number of groups a user belongs to, or number of users in a group.
- **Regular graph**: every node has the same degree k (**k-regular**); can be connected or disconnected. Complete graphs are a special case.

Traversal algorithms

Used, e.g., for a web crawler collecting topic-relevant pages starting from a seed page:

- **Depth-First Search (DFS)**: from v_i , pick one neighbor $v_j \in N(v_i)$ and recurse into DFS from v_j before exploring v_i 's other neighbors.
- **Breadth-First Search (BFS)**: visit all immediate neighbors first, then expand level by level.
- **Random Walk** (as defined in Week 1).

Minimum Spanning Tree — Prim's Algorithm

(The only MST algorithm covered — no Kruskal's in these slides.)

1. Pick a random start node; add it to the MST.
 2. Repeatedly choose, among all edges with exactly one endpoint already in the MST and one endpoint outside it, the **minimum-weight** such edge; add it (and its new endpoint) to the tree.
 3. Repeat until the tree spans the whole graph.
-

Week 3 — Centrality Measures I

“**Network Measures**” chapter. Network measures answer three questions in social network analysis:

- **Centrality** — who are the influential/central individuals?
- **Reciprocity & transitivity** — what interaction patterns are common among friends? (covered in Week 4)
- **Similarity** — who are like-minded users, and how do we find them? (covered in Week 4)

This week covers **centrality in terms of who you are connected to**: degree, eigenvector, Katz, and PageRank.

1. Degree Centrality

$$C_d(v_i) = d_i$$

Directed graphs — three variants:

- $C_d(v_i) = d_i^{in} \rightarrow$ interpreted as **prestige**
- $C_d(v_i) = d_i^{out} \rightarrow$ interpreted as **gregariousness**
- $C_d(v_i) = d_i^{in} + d_i^{out} \rightarrow$ combined (in-degree is most commonly used in practice)

Normalizations:

- By max possible degree: $C_d^{norm}(v_i) = \frac{d_i}{n-1}$
- By max observed degree: $C_d^{max}(v_i) = \frac{d_i}{\max_j d_j}$
- By degree sum: $C_d^{sum}(v_i) = \frac{d_i}{\sum_j d_j} = \frac{d_i}{2m}$

Intuition: raw popularity/connectivity — doesn't account for who you're connected to, only how many.

2. Eigenvector Centrality

Motivation (Phillip Bonacich): having more friends doesn't by itself make you important — having more important friends is a stronger signal. Generalizes degree centrality by weighting each neighbor's contribution by that neighbor's own importance.

$$c_e(v_i) = \frac{1}{\lambda} \sum_{j=1}^n A_{j,i} c_e(v_j)$$

Matrix form: $\lambda \mathbf{C}_e = A^T \mathbf{C}_e$ — i.e., $\mathbf{C}_e$ is an **eigenvector** of A^T (or A , if undirected) with eigenvalue λ .

Which eigenvalue? Solved via the **power method**: start from an initial guess $C_e(0)$ (e.g., all 1's) and iterate $C_e(t) = (A^T)^t C_e(0)$. As $t \rightarrow \infty$ this converges to the eigenvector for the **largest** eigenvalue λ_1 .

Perron-Frobenius Theorem: for a [strongly] connected graph, the adjacency matrix has a unique largest positive eigenvalue λ_{max} whose eigenvector has all-positive components — this guarantees eigenvector centralities are all comparable (same sign).

Procedure: compute eigenvalues of $A \rightarrow$ take the largest, $\lambda \rightarrow$ its eigenvector is $\mathbf{C}_e$ (all components positive by Perron-Frobenius) $\rightarrow$ those components are the centralities (in practice, computed via the power method).

Limitation (motivates Katz): in **directed graphs**, eigenvector centrality only propagates over outgoing edges — a node with no incoming edges from “important” nodes can get **zero** centrality even with many edges (e.g., any node in a DAG).

3. Katz Centrality

Motivation (Elihu Katz): fixes eigenvector centrality's zero-centrality problem by adding a constant **bias term** β , giving every node a baseline centrality regardless of incoming links.

$$C_{Katz}(v_i) = \alpha \sum_{j=1}^n A_{j,i} C_{Katz}(v_j) + \beta$$

(first term = the eigenvector-style “controlling term” propagated from neighbors; β = the constant “bias term” every node gets regardless of neighbors)

Vector form: $\mathbf{C}_{Katz} = \alpha A^T \mathbf{C}_{Katz} + \beta \mathbf{1}$

Closed form: $\mathbf{C}_{Katz} = \beta (I - \alpha A^T)^{-1} \cdot \mathbf{1}$

- $\alpha = 0 \rightarrow$ every node gets centrality $= \beta$ (pure bias, no propagation).
- As α grows, the effect of β shrinks and the measure behaves more like eigenvector centrality.
- **Convergence constraint:** $\alpha < 1/\lambda$, where λ is the largest eigenvalue of A^T .

4. PageRank

Motivation: fixes a remaining flaw in Katz — Katz passes a node's full centrality to **every** outgoing neighbor (unrealistic: not everyone known by a celebrity is themselves famous). PageRank **divides** the passed centrality by the node's out-degree, so each neighbor gets only a fraction.

$$C_p(v_i) = \alpha \sum_{j=1}^n A_{j,i} \frac{C_p(v_j)}{d_j^{out}} + \beta$$

With $D = \text{diag}(d_1^{out}, \dots, d_n^{out})$ (defined only over nodes with $d_j^{out} > 0$):

Closed form: $\mathbf{C}_p = \beta(\mathbf{I} - \alpha \mathbf{A}^T \mathbf{D}^{-1})^{-1} \cdot \mathbf{1}$

- Convergence: $\alpha < 1/\lambda$, where λ is the largest eigenvalue of $\mathbf{A}^T \mathbf{D}^{-1}$.
- **Undirected graphs:** the largest eigenvalue of $\mathbf{A}^T \mathbf{D}^{-1}$ is always $\lambda = 1$, so you just need $\alpha < 1$.
- **Random-surfer interpretation:** PageRank equals the stationary distribution of a random walk over the graph — repeated power-iteration steps starting from a uniform distribution converge to the PageRank vector.
- **Original purpose:** rank a search index by (1) building an index of pages per query term, (2) matching query text to pages, (3) ranking matches by importance (PageRank) as well as relevance.

Progression to remember: Degree $\rightarrow$ Eigenvector (weight by neighbor importance) $\rightarrow$ Katz (add baseline to fix directed-graph zeros) $\rightarrow$ PageRank (divide by out-degree so influence doesn't multiply for free).

Week 4 — Centrality Measures II & Similarity

Continues directly from Week 3.

1. Betweenness Centrality

Intuition: centrality in terms of how you connect others — an “information broker” score. If messages between every pair of nodes travel via shortest paths with equal probability, betweenness measures how many of those messages pass through a given node on average.

$$C_b(v_i) = \sum_{s \neq t \neq v_i} \frac{\sigma_{st}(v_i)}{\sigma_{st}}$$

where σ_{st} = number of shortest paths from s to t , and $\sigma_{st}(v_i)$ = number of those passing through v_i .

High-betweenness nodes: (a) most messages pass through them, (b) removing them most disrupts communication.

Normalization: max possible value is $(n-1)(n-2)$, giving $C_b^{norm}(v_i) = \frac{C_b(v_i)}{2\binom{n-1}{2}}$.

2. Closeness Centrality

Intuition: centrality in terms of how fast you can reach others — central nodes have a small average shortest-path distance to everyone else.

$$C_c(v_i) = \frac{1}{\bar{l}_{v_i}}, \quad \bar{l}_{v_i} = \frac{1}{n-1} \sum_{v_j \neq v_i} l_{i,j}$$

Key exam point: on directed graphs, distances aren’t symmetric ($d(A, C) \neq d(C, A)$ in general), so closeness rankings can differ substantially between the directed and undirected versions of the same graph.

3. Group Centrality: Cliques and Cores

- **Clique:** a maximal subset of vertices where every member connects to every other member (“maximal” = no vertex can be added while preserving full connectedness).
- **k-core:** a maximal subset of vertices where each vertex connects to at least k others within that subset (nested cores of increasing k = increasingly tightly-knit inner groups).

4. Friendship Patterns: Transitivity & Reciprocity

Transitivity: “a friend of my friend is my friend” — if $v_1 \rightarrow v_2$ and $v_2 \rightarrow v_3$, transitivity implies $v_1 \rightarrow v_3$. More transitivity $\rightarrow$ denser graph, closer to complete.

Global clustering coefficient (transitivity across the whole undirected graph):

$$C = \frac{|\text{Closed Paths of Length 2}|}{|\text{Paths of Length 2}|} = \frac{6 \times (\text{Number of Triangles})}{|\text{Paths of Length 2}|}$$

(each triangle contributes 6 closed length-2 paths — traversable from any of its 3 vertices, in 2 directions each.)

Local clustering coefficient (transitivity at a single node — how interconnected v_i 's neighbors are):

$$C(v_i) = \frac{\text{Number of Connected Pairs of Neighbors of } v_i}{\text{Number of Pairs of Neighbors of } v_i}, \quad \text{denominator} = \binom{d_i}{2} = \frac{d_i(d_i - 1)}{2}$$

Reciprocity — a 2-node version of transitivity (“if you become my friend, I’ll be yours”), for directed graphs, counting mutual (closed length-2) edges:

$$R = \frac{\sum_{i,j, i < j} A_{i,j}A_{j,i}}{|E|/2} = \frac{1}{m} \text{Tr}(A^2)$$

5. Similarity (Structural Equivalence)

Structural equivalence: two nodes are similar if they share a large neighborhood (analogy: two brothers share sisters/parents — similar; two random strangers don’t).

- **Raw vertex similarity:** $\sigma(v_i, v_j) = |N(v_i) \cap N(v_j)|$ — problem: directly connected nodes with no common neighbor score 0.

- **Jaccard similarity:** $\sigma_{\text{Jaccard}}(v_i, v_j) = \frac{|N(v_i) \cap N(v_j)|}{|N(v_i) \cup N(v_j)|}$

- **Cosine similarity:** $\sigma_{\text{Cosine}}(v_i, v_j) = \frac{|N(v_i) \cap N(v_j)|}{\sqrt{|N(v_i)||N(v_j)|}}$ (tends to score higher than

Jaccard for the same overlap, since it doesn’t penalize union size as harshly)

- **Similarity significance:** compares observed similarity to what random neighbor-selection would predict. For degrees d_i, d_j , expected common neighbors = $d_i d_j / n$. Writing overlap via the adjacency matrix, $\sigma(v_i, v_j) = \sum_k A_{i,k} A_{j,k}$, and subtracting the random baseline reduces algebraically to **n times the covariance** between adjacency rows A_i and A_j :

$$\sigma_{significance}(v_i, v_j) = \sum_k (A_{i,k} - \bar{A}_i)(A_{j,k} - \bar{A}_j)$$

Normalizing by the product of standard deviations gives the **Pearson correlation** of the two adjacency rows:

$$\sigma_{pearson}(v_i, v_j) = \frac{\sum_k (A_{i,k} - \bar{A}_i)(A_{j,k} - \bar{A}_j)}{\sqrt{\sum_k (A_{i,k} - \bar{A}_i)^2} \sqrt{\sum_k (A_{j,k} - \bar{A}_j)^2}} \in [-1, 1]$$

Key link: structural similarity between two nodes can be computed exactly like a statistical correlation between their adjacency vectors.

Week 5 — Community Detection

What is a community?

A group of individuals sharing common interests, characteristics, or interactions (other names: group, cluster, cohesive subgroup, module).

Why analyze communities? Reveals how users group by interest → clearer global view of interactions (e.g., detecting polarization); on the web, communities often correspond to topically related pages.

Formation types:

- **Explicit (emic)** — formed via user subscriptions.
- **Implicit (etic)** — formed through interaction patterns (e.g., grouping everyone who calls Canada from the US, for a promotion).

Overlapping vs. disjoint communities: nodes may belong to multiple communities (overlapping) or exactly one (disjoint). Ideal = fully disjoint communities with zero inter-community edges; in practice algorithms just try to maximize decoupling.

Community analysis has three subtasks: (1) detection — finding implicit communities, (2) evolution — how communities change over time, (3) evaluation — assessing quality of detected communities.

Community detection vs. clustering: clustering (e.g., k-means) works on a distance/similarity matrix and, applied to an adjacency matrix, only sees the ego-centric neighborhood. Community detection works directly on graph structure, using graph-native concepts like k-clique, quasi-clique, and edge betweenness.

Zachary's Karate Club (canonical example): 34 members interacting over two years; a real split occurred between the administrator (node 34) and instructor (node 1), with one group leaving to start a new club. Community detection algorithms recover this exact real-world split, which is why it's the standard benchmark ([week5/karate.gml](#)).

Algorithm families

1. Member-based — group nodes by node-level characteristics:

- Degree: similar-degree nodes → same community (cliques)
- Reachability: nodes close together (small shortest-path distance) → same community (k-cliques, k-clubs, k-clans)
- Similarity: similar-neighborhood nodes → same community (Jaccard, cosine)

2. **Group-based** — find communities with a global group property:

- Modular: modularity maximization
- Hierarchical: agglomerative or divisive clustering

Modularity

Intuition: given only a graph's degree sequence (not its actual edges), you can compute the expected number of edges between any two nodes under a random-graph null model. Real networks deviate from this baseline — modularity measures that deviation, and modularity maximization finds the partition maximizing it.

Expected edge count between v_i, v_j (degrees d_i, d_j , m total edges): $\frac{d_i d_j}{2m}$.

$$Q = \frac{1}{2m} \sum_{i=1}^n \sum_{j=1}^n \left(A_{ij} - \frac{d_i d_j}{2m} \right) \delta(P_i, P_j)$$

$\delta(P_i, P_j) = 1$ iff i, j are in the same community. Q ranges roughly $[-1, 1]$; higher = stronger community structure. **Exact modularity maximization is NP-hard**, so heuristics (like Louvain) are needed at scale.

The Louvain Method

Blondel, Guillaume, Lambiotte & Lefebvre (2008) — scales to **billions of nodes**. Each pass has two phases:

Phase 1 — Local modularity optimization

1. Start with every node in its own singleton community.
2. For each node i and each neighboring community, compute the modularity gain ΔQ of moving i into that community.
3. Move i to the neighbor community with the **maximum positive gain**; if none is positive, leave i in place.
4. Repeat over all nodes until no node moves (local modularity maximum).

$$\Delta Q = \left[\frac{\Sigma_{in} + 2k_{i,in}}{2m} - \left(\frac{\Sigma_{tot} + k_i}{2m} \right)^2 \right] - \left[\frac{\Sigma_{in}}{2m} - \left(\frac{\Sigma_{tot}}{2m} \right)^2 - \left(\frac{k_i}{2m} \right)^2 \right]$$

(Σ_{in} = sum of link weights inside community C ; Σ_{tot} = sum of link weights incident to nodes in C ; k_i = weighted degree of node i ; $k_{i,in}$ = weight from i into C .)

Note: the result is order-dependent in path but empirically stable in final modularity achieved — order mainly affects computation time, not quality.

Phase 2 — Community aggregation

- Build a smaller network where each node = one community from Phase 1.
- New edge weight = sum of edge weights between the two communities' original nodes.
- Self-loops = sum of weights within each community.
- Re-apply Phase 1 to this smaller network, repeat.

Properties: creates a natural hierarchy of communities; typically converges in well under 10 passes (Karate Club: 34→6→4 communities in 3 passes, final $Q = 0.42$, matching all reference methods); scales to 100M+ nodes (paper benchmark: 118M nodes / 1B links in 152 minutes, $Q = 0.984$) — storage, not compute, is the real bottleneck. Beats prior greedy methods (CNM, Pons-Latapy, Wakita-Tsurumi) on both speed and modularity across all tested network sizes, and is less prone to degenerate “super-communities.”

Hierarchical Community Detection

Builds a full hierarchy rather than one flat partition:

- **Agglomerative:** start with singleton communities, merge upward.
- **Divisive:** start with the whole network as one community, split downward.

Girvan-Newman algorithm (divisive):

1. Compute **edge betweenness** for every edge (number of shortest paths in the whole graph passing through it — high betweenness $\approx$ a bridge between communities).
 2. Remove the highest-betweenness edge.
 3. Recalculate betweenness for affected edges.
 4. Repeat until all edges are removed, producing a full dendrogram — each connected component at any stage is a community.
-

Week 6 — Recommendation in Social Media

Chapter 9 (Part 1) of the textbook: “Recommendation in Social Media.” This part covers content-based filtering; collaborative filtering is Part 2 (not yet uploaded).

When does the recommendation problem occur?

Users face **information overload**: many choices, no obvious advantage between them, and not enough time/knowledge to evaluate every option (but they don’t want to miss out on good stuff). Traditional fixes — asking friends, trusted third parties, hiring experts, searching, or “following the crowd” (top- n /best-seller lists) — can be automated with a **recommender algorithm**, i.e. a **recommender system**.

Goal of recommendation: produce a short list of items that fits a user’s interests.

Recommendation vs. search: a search engine returns results matching an explicit query, ranked by relevance to that query — the same query gives the same ranked list to everyone. A recommender’s results are **personalized to the user**, without requiring an explicit query.

Main idea

Use historical data — a user’s own past preferences, or similar users’ past preferences — to predict future likes, on the assumption that preferences are stable and change only smoothly over time.

Formally, a recommender system takes a set of users U and a set of items I and learns a function

$$f: U \times I \rightarrow \mathbb{R}$$

scoring how much a given user would like a given item.

Challenges of recommender systems

- **Cold-start problem**: a new user (or item) has no history, so there’s nothing to infer preferences from.
- **Data sparsity**: historical/prior information is insufficient system-wide (not specific to one user/item, unlike cold-start).
- **Attacks**: adversaries push ratings up by creating fake users.
- **Privacy**: using one user’s private info to generate recommendations for others.

- **Explanation:** recommendations are often given with no explanation of why an item was recommended — motivates **explainable recommender systems**, one facet of “Trustworthy AI” alongside fairness, privacy, robustness, accountability, and safety.

Two classical approaches

1. **Content-based algorithms** — recommend items similar to what the user liked before, based on item/user descriptions.
2. **Collaborative filtering** — recommend based on what similar users liked (Part 2, not covered in this deck).

Content-based methods

Assumption: a user’s interest should match the description of items recommended to them — the more similar an item’s description is to the user’s interest profile, the more likely the user finds it interesting.

Goal: find the similarity between the user (profile) and all existing items.

Example: a book database has structured fields (Title, Genre, Author, Type, Price, Keywords); a user profile is built as a similar structured record summarizing the genres/authors/keywords the user tends to interact with. Items whose descriptions best match the profile get recommended (e.g., Amazon product recommendations built from browsing/purchase history).

Algorithm outline:

1. Describe the items to be recommended.
2. Create a profile of the user describing the types of items they like (often built/updated automatically from feedback on previously shown items).
3. Compare items with the user profile to determine what to recommend.

Illustration: if user u_3 interacted with item i_2 (basic seat: [RFIC-A, EMD-A, BS-Basic]) and item i_3 (premium seat: [RFIC-A, EMD-A, PR-Premium]) shares most of the same characteristics, i_3 gets recommended to u_3 via content-based filtering even without u_3 ever interacting with it directly.

Vectorizing users and items

Represent both user profiles and item descriptions as vectors over a shared set of k keywords, then rank items by similarity to the user vector:

$$I_j = (i_{j,1}, i_{j,2}, \dots, i_{j,k}), \quad U_i = (u_{i,1}, u_{i,2}, \dots, u_{i,k})$$

$$\text{sim}(U_i, I_j) = \cos(U_i, I_j) = \frac{\sum_{l=1}^k u_{i,l} i_{j,l}}{\sqrt{\sum_{l=1}^k u_{i,l}^2} \sqrt{\sum_{l=1}^k i_{j,l}^2}}$$

Recommend the top- r most similar items to the user.

Text representation: Bag-of-Words & Vector Space Model

Documents (item descriptions / profiles built from text) are commonly transformed into vectors — the **Bag-of-Words / Vector Space Model** representation — so they can be manipulated with linear algebra.

For a document corpus D , each document i becomes $d_i = (w_{1,i}, w_{2,i}, \dots, w_{N,i})$, where $w_{j,i}$ is the weight of word j in document i . Simplest weighting: $w_{j,i} = 1$ if word j appears in document i , else 0. A richer option is raw **frequency** (count of occurrences), and richer still is **TF-IDF**.

Worked example — documents d_1 = “social media mining”, d_2 = “social media data”, d_3 = “financial market data”; dictionary (social, media, mining, data, financial, market) gives the binary term-presence vectors:

	social	media	mining	data	financial	market
d_1	1	1	1	0	0	0
d_2	1	1	0	1	0	0
d_3	0	0	0	1	1	1

TF-IDF (Term Frequency - Inverse Document Frequency)

$$w_{j,i} = tf_{j,i} \times idf_j, \quad idf_j = \log_2 \frac{|D|}{|\{\text{document} \in D \mid j \in \text{document}\}|}$$

$tf_{j,i}$ = frequency of word j in document i ; idf_j down-weights words that appear in many documents (less discriminative) and up-weights rare, distinctive words.

Worked example (frequency form): d_1 has 100 words; “apple” appears 10 times in d_1 and in no other of the $|D| = 20$ documents, while “orange” appears 20 times in d_1 but also appears in all 20 documents:

$$tf-idf(\text{"apple"}, d_1) = 10 \times \log_2 \frac{20}{1} = 43.22$$

$$tf-idf(\text{"orange"}, d_1) = 20 \times \log_2 \frac{20}{20} = 0$$

“Apple” is distinctive to d_1 and scores high; “orange” appears everywhere and is scored 0 — despite the higher raw frequency, it carries no discriminating power.

Worked example (binary $tf \in \{0, 1\}$ form, continuing the social-media-mining example): with $|D| = 3$, $idf_{\text{social}} = idf_{\text{media}} = idf_{\text{data}} = \log_2(3/2) = 0.584$ (each appears in 2 of 3 docs) and $idf_{\text{mining}} = idf_{\text{financial}} = idf_{\text{market}} = \log_2(3/1) = 1.584$ (each appears in only 1 doc), giving the TF-IDF matrix:

	social	media	mining	data	financial	market
d_1	0.584	0.584	1.584	0	0	0
d_2	0.584	0.584	0	0.584	0	0
d_3	0	0	0	0.584	1.584	1.584

Content-based recommendation algorithm (Algorithm 9.1)

Require: user i 's profile info, item descriptions for items $j \in \{1, \dots, n\}$, k keywords, r = number of recommendations.

1. $U_i = (u_1, \dots, u_k)$ = user i 's profile vector.
2. $\{l_j\}_{j=1}^n$ = item description vectors.
3. $s_{i,j} = \text{sim}(U_i, l_j)$ for $1 \leq j \leq n$.
4. Return the top r items with maximum similarity $s_{i,j}$.

(Dis)advantages of content-based recommendation

- Does **not** leverage information from other, similar users (no “wisdom of the crowd” effect) — this is exactly what collaborative filtering (Part 2) adds.
- **Lack of novelty:** tends to recommend items very similar to what the user already likes, rarely surprising them.
- **Useful for cold-start:** works even for a brand-new item as soon as it has a description — unlike collaborative filtering, it doesn’t need other users to have already rated/interacted with it.

Case Recommender: an example open-source framework

[week6/caseRecommender.pdf](#) is the RecSys '18 paper for **Case Recommender**, an open-source Python framework (`pip install caserecommender` , MIT license) implementing both rating-prediction and item-recommendation scenarios:

- **Neighborhood-based (NB)** and **matrix factorization (MF)** models for both collaborative and content-based filtering.
 - **Clustering** algorithms (PaCo, KMedoids) to pre-process data for recommenders, and **ensemble** techniques (e.g., BPR Learning, tag-based, average-based) to combine multiple recommenders — features the paper highlights as rarer among competing toolkits (EasyRec, Mahout, LensKit, RiVal, MyMediaLite, Crab).
 - 15+ (dis)similarity metrics (cosine, Pearson, etc.) via SciPy.
 - Evaluation: RMSE/MAE for rating prediction; prec@N, recall@N, MAP@N, NDCG@N for item recommendation; K-fold cross-validation, Shuffle Sample, All-But-One protocol, and significance tests (Wilcoxon signed-rank, paired t-test).
-

Dataset Index

Week	Path	Represents
1	<code>week1/nodes.csv</code> , <code>week1/edges.csv</code>	9-node weighted “propagation cost” graph (Twitter example); reused in Week 2 for MST practice
2	<code>week2/nodesMST.csv</code> , <code>week2/edgesMST.csv</code>	Same 9-node structure, different weights — practice graph for Prim’s algorithm
3	<code>week3/betweenness/</code>	Nodes A-I, undirected — pairs with Week 4’s betweenness centrality
3	<code>week3/eigenvector/</code>	3-node path A-B-C — matches the eigenvector centrality worked example
3	<code>week3/katz/</code> (+ <code>katz-centrality-1.0.1.nbm</code>)	5-node graph V1-V5 matching the Katz centrality worked example; <code>.nbm</code> is a NetBeans/Gephi plugin binary implementing Katz centrality
3	<code>week3/PageRank/</code>	7-node directed graph A-G matching the PageRank/Markov-chain worked example
4	<code>week4/clustering/</code>	4-node graph (triangle + pendant) — hand-computable local clustering coefficient example
5	<code>week5/Community detection/</code>	9-node, 15-edge toy graph used for both the modularity-maximization example (communities {1,2,3,4} vs {5,6,7,8,9}) and the Girvan-Newman walkthrough
5	<code>week5/karate.gml</code>	Zachary’s Karate Club — 34 nodes, 78 edges, the canonical community-detection benchmark
6	<code>week6/SMM-Slides-ch9-part1.pdf</code>	Chapter 9 (Part 1) slides — content-based recommendation, TF-IDF, Algorithm 9.1
6	<code>week6/caseRecommender.pdf</code>	RecSys '18 paper on the Case Recommender Python framework (no graph dataset this week)

Concept Progression Cheat Sheet

Graph Basics (W1) → Spanning Trees / Prim's (W2)
 ↓
 Degree → Eigenvector → Katz → PageRank centrality (W3)

(each fixes a flaw in the previous one)

↓

Betweenness / Closeness / Cliques / Transitivity / Similarity (W4)

↓

Modularity → Louvain → Hierarchical (Girvan-Newman) community detection (W5)

↓

Recommendation: Content-based filtering (TF-IDF + cosine similarity) (W6)

(Part 2 – Collaborative filtering – to follow)